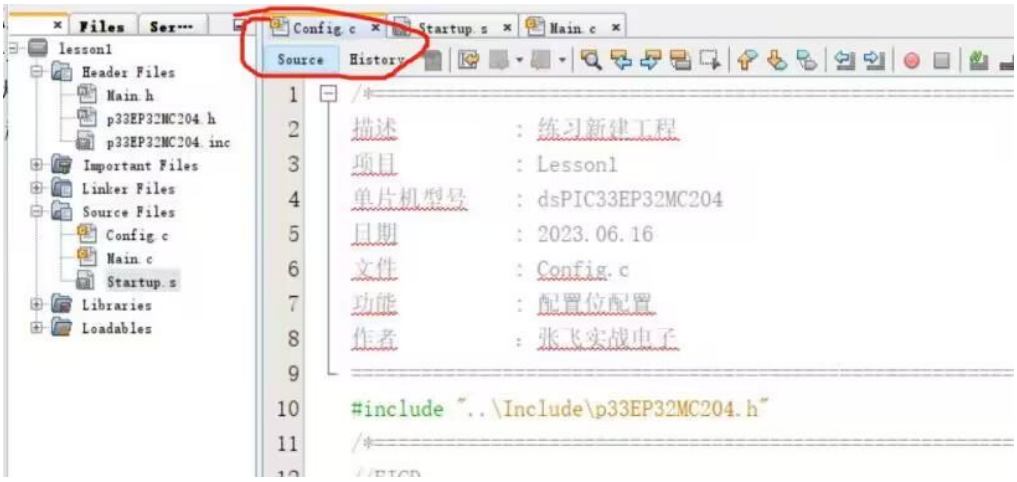
# 22-单片机 4

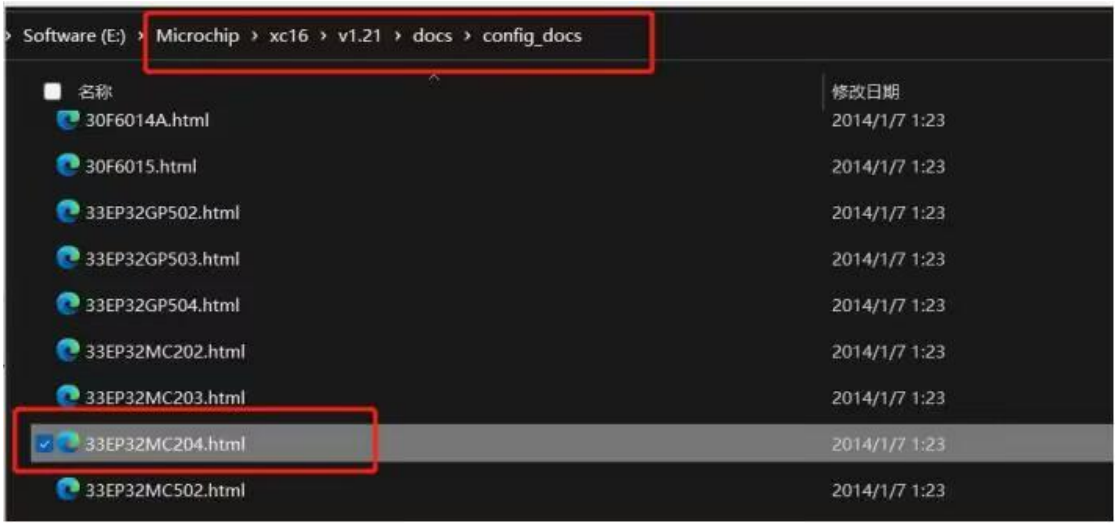
大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上节课，我把这 6 个配置位寄存器讲完了。今天我们讲一下，前面的配置如何写代码。我们要想使用单片机，是需要先把配置位配置好的。所以，我们之前要了解写的一段延时代码的时长，是配置位配置好之后，再来看对应的时钟是多少。它们之间呢，是有一些联系的哈。那么，配置位对应的功能我们是清楚了。但是，具体要怎么配置，我们下面就一起来看看。

配置位的配置呢，我们一般是专门用一个 C 文件来写。这里大家要注意啊，我讲的有点跳，先把配置位寄存器文件写好后，我们再回去继续讲启动文件。

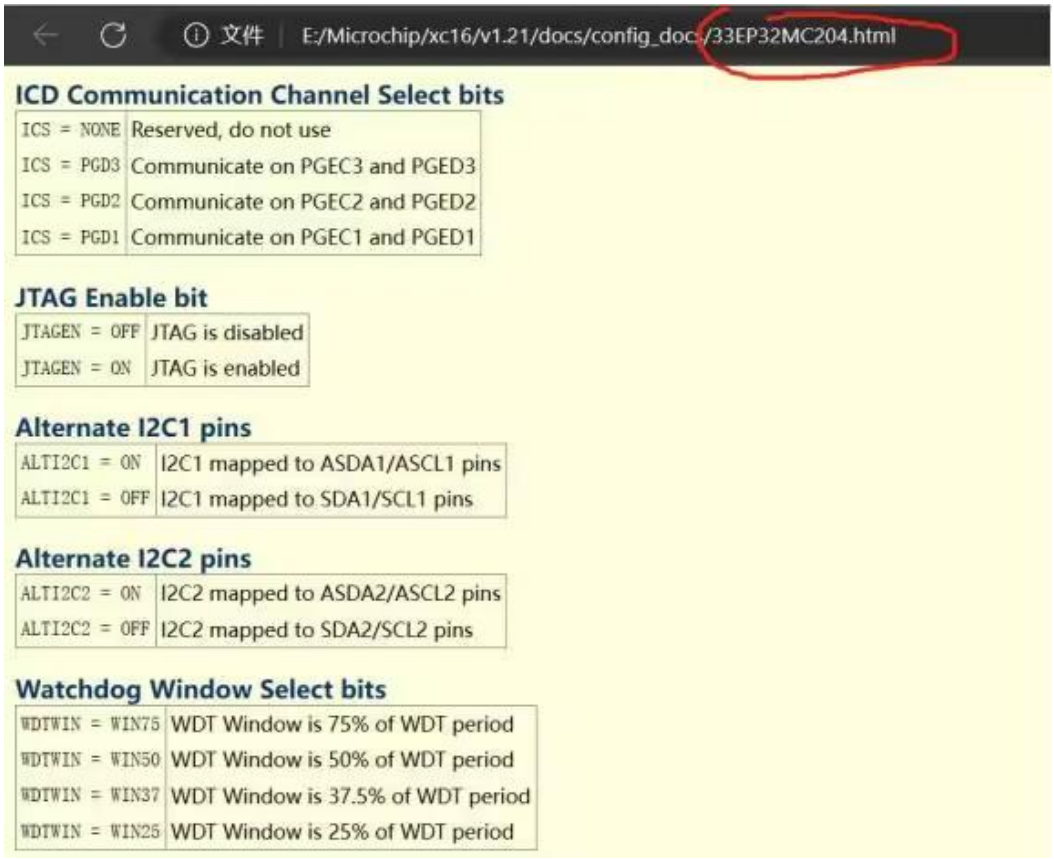



我把讲的这部分代码上传了，在 Lesson1 里面有 Config.c 文件。在这个 Config.c 文件中，对我们上次课介绍过的寄存器对应位进行设置。实际上，官方给我们提供了一个配置位配置参考文档。




这个文档的参考路径如上图所示。在这个路径下，我们可以找到我们使用的芯片型号对应的文档。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。




打开之后，文档中会有这些位的全部配置方式的罗列。我们呢，其实把需要配置的功能给复制过来就可以了。

```
#include "..\Include\p33EP32MC204.h"

//FICD
#pragma config ICS = PGD2
#pragma config JTAGEN = OFF
```

只不过呢，配置位的配置呢，需要加上 #pragma config 来告知编译器。在 #pragma config 加上复制来的值，就可以对配置寄存器进行配置了。上面两个配置位呢，是对仿真引脚及 JTAG 的功能配置。其它的五个配置寄存器，就按照我们上次课讲的进行选择即可。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
// FPOR
#pragma config ALTI2C1 = OFF
#pragma config ALTI2C2 = OFF
#pragma config WDTWIN = WIN25

// FWDT
#pragma config WDTPOST = PS32768
#pragma config WDTPRE = PR128
#pragma config PLLKEN = ON
#pragma config WINDIS = OFF
#pragma config FWDTEN = OFF

// FOSC
#pragma config POSCMD = NONE
#pragma config OSCIOFNC = OFF
#pragma config IOL1WAY = OFF
#pragma config FCKSM = CSDCMD

// FOSCSEL
#pragma config FNOSC = FRCPLL
#pragma config PWMLOCK = ON
#pragma config IESO = OFF

// FGS
#pragma config GWRP = ON
#pragma config GCP = ON
/*==========Copyr
```

咱们重点呢，是选择 FRCPLL 来产生系统时钟，然后选择代码保护。到这里 Config.c 就完了。

问：这个文件是在哪个程序里调用执行的？
答：这个文件不执行，只是定义数据，只参与编译。

问：那也就是说，我只要在这里配置好，就不需要在程序里执行了？
答：是的，这只是特殊数据的定义，编译后并不是指令。

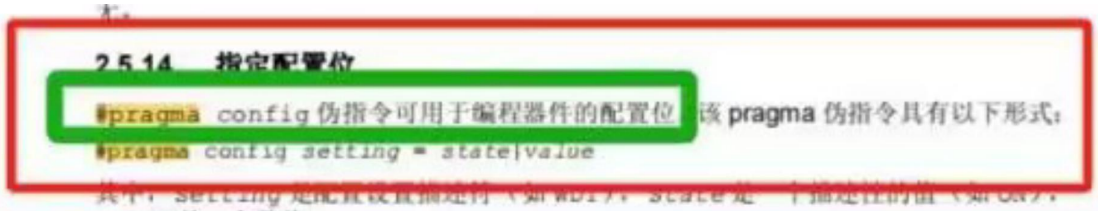
问：包括 ON 在内的参数是在哪个文件里有定义？
答：这个是编译器本身内部定义的，和用户无关，如何定义的，在哪定义的不得而知。

问：那就是说#pragma config 加上寄存器的值配置上面的 5 个寄存器了吗？哪些宏是定义好的？其他芯片我也对应着替换就行了吧？
答：同系列的单片机的其他芯片可以替换。

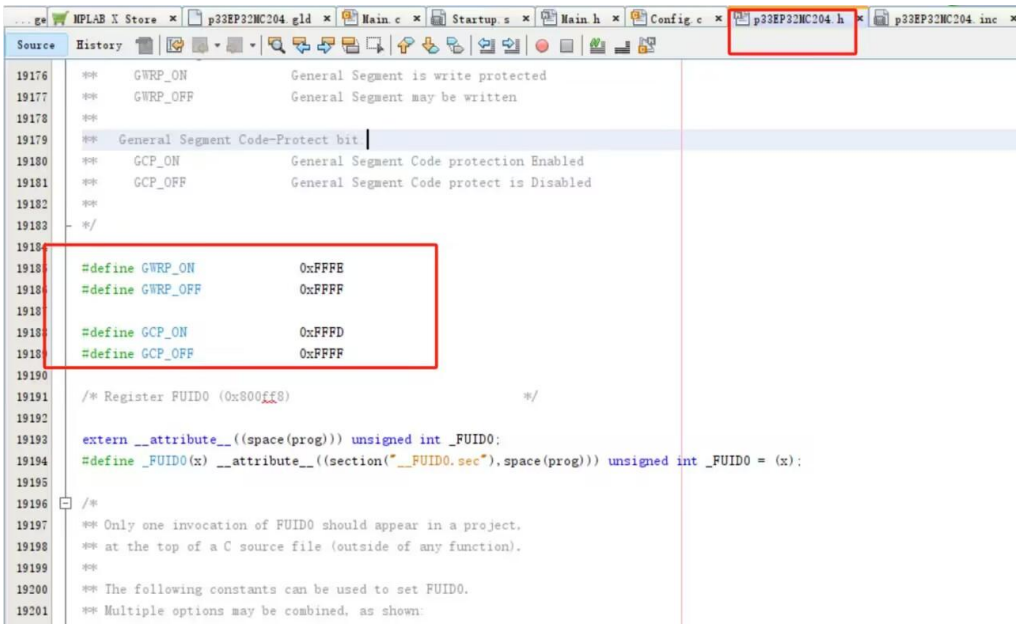
问：如果修改上面的代码，单片机会不会执行对应的修改？
答：会的。

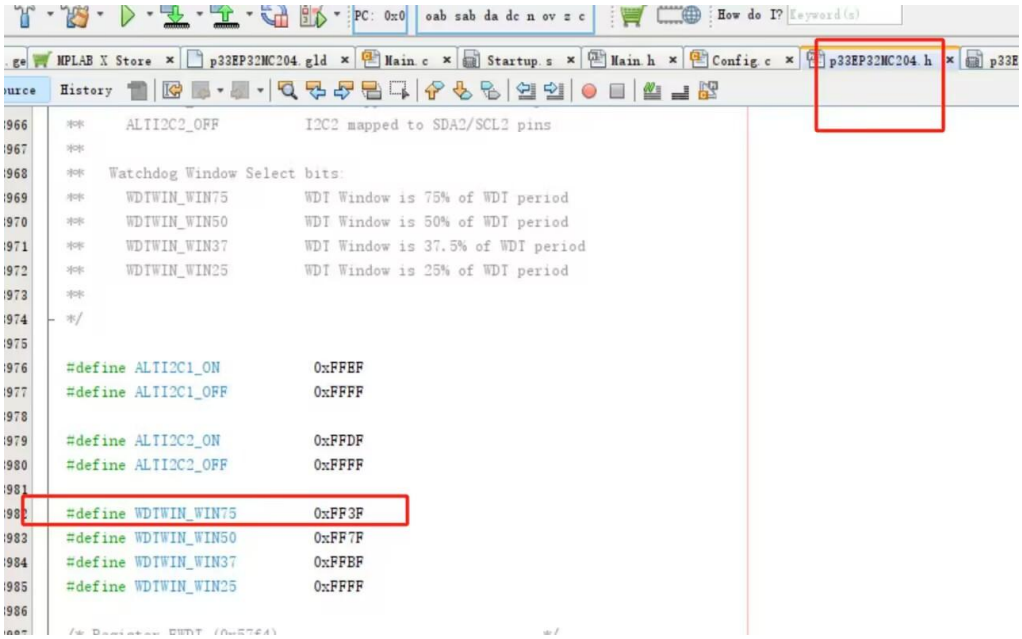
问：#pragma config 这两个词，没解释明白？


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



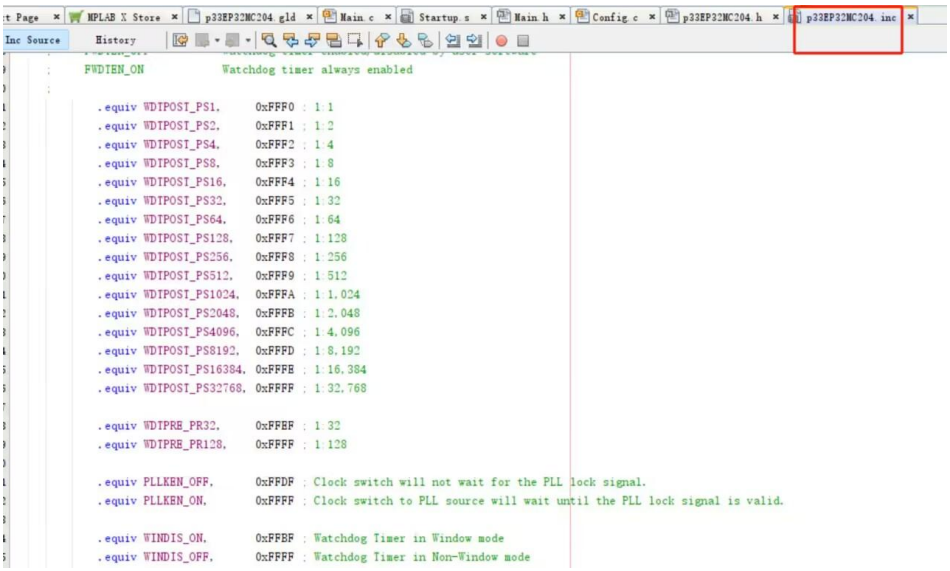

答：





本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



```
FWDTEN_ON           Watchdog timer always enabled

  .equiv WDTPOST_PS1,     0xFFF0 ; 1:1
  .equiv WDTPOST_PS2,     0xFFF1 ; 1:2
  .equiv WDTPOST_PS4,     0xFFF2 ; 1:4
  .equiv WDTPOST_PS8,     0xFFF3 ; 1:8
  .equiv WDTPOST_PS16,    0xFFF4 ; 1:16
  .equiv WDTPOST_PS32,    0xFFF5 ; 1:32
  .equiv WDTPOST_PS64,    0xFFF6 ; 1:64
  .equiv WDTPOST_PS128,   0xFFF7 ; 1:128
  .equiv WDTPOST_PS256,   0xFFF8 ; 1:256
  .equiv WDTPOST_PS512,   0xFFF9 ; 1:512
  .equiv WDTPOST_PS1024,  0xFFFA ; 1:1,024
  .equiv WDTPOST_PS2048,  0xFFFB ; 1:2,048
  .equiv WDTPOST_PS4096,  0xFFFC ; 1:4,096
  .equiv WDTPOST_PS8192,  0xFFFD ; 1:8,192
  .equiv WDTPOST_PS16384, 0xFFFE ; 1:16,384
  .equiv WDTPOST_PS32768, 0xFFFF ; 1:32,768

  .equiv WDTPRE_PR32,     0xFFEF ; 1:32
  .equiv WDTPRE_PR128,    0xFFFF ; 1:128

  .equiv PLLKEN_OFF,      0xFFDF ; Clock switch will not wait for the PLL lock signal.
  .equiv PLLKEN_ON,       0xFFFF ; Clock switch to PLL source will wait until the PLL lock signal is valid.

  .equiv WINDIS_ON,       0xFFBF ; Watchdog Timer in Window mode
  .equiv WINDIS_OFF,      0xFFFF ; Watchdog Timer in Non-Window mode
```

有些符号在头文件中定义好了。
好，我们继续往下讲了。

```
mov     #4, W1    ;1个指令周期
clr     W0        ;1个指令周期 * 4
dec     W0, W0    ;1个指令周期 * 65536 * 4
bra     NZ, $-2   ;未跳转是1个指令周期，跳转是4个指令周期 (65535 * 4 + 1) * 4
dec     W1, W1    ;1个指令周期 * 4
bra     NZ, $-8   ;未跳转是1个指令周期，跳转是4个指令周期  3*4 + 1
```

咱们继续回到这段代码啊
配置位配置好了之后，下面呢，我们需要来看下此时默认的指令时钟频率是多少？

**公式 9-2：　　Fosc 计算**

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2) \times 2(PLLPOST+1)}\right)$$

其中：
$N1 = PLLPRE + 2$
$N2 = 2 \times (PLLPOST + 1)$
$M = PLLDIV + 2$

上面的内容在振荡器章节。

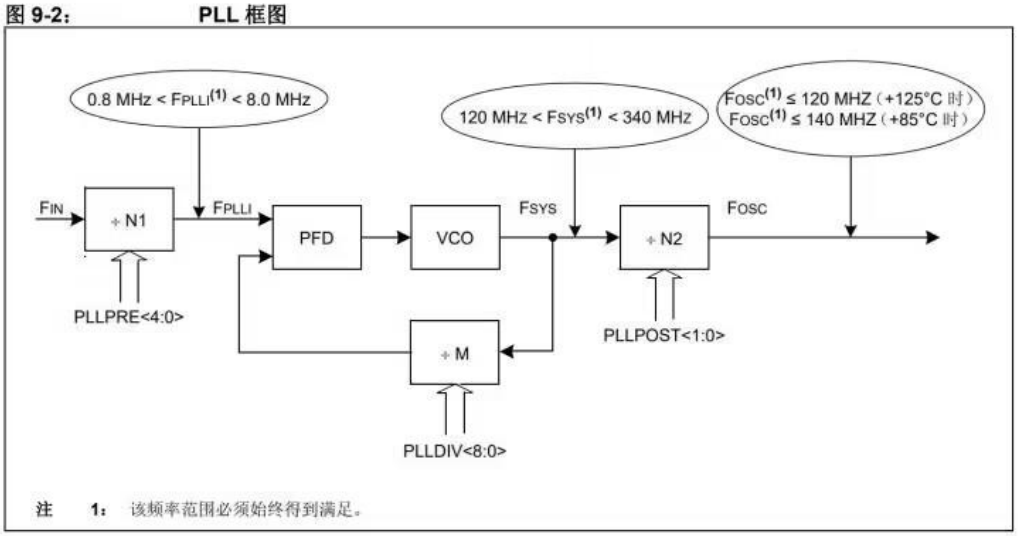
问：这一部分的配置是在.S 文件前执行的吗？
答：不执行，只参与编译。

问：寄存器的配置是在.S 文件执行之前就完成了吗？
答：寄存器配置是在代码烧录(下载)时完成。

大家来看下这个公式啊。这个公式描述了，一个低频时钟经过了 PLL 倍频之后，变为高频时钟的计算方法。

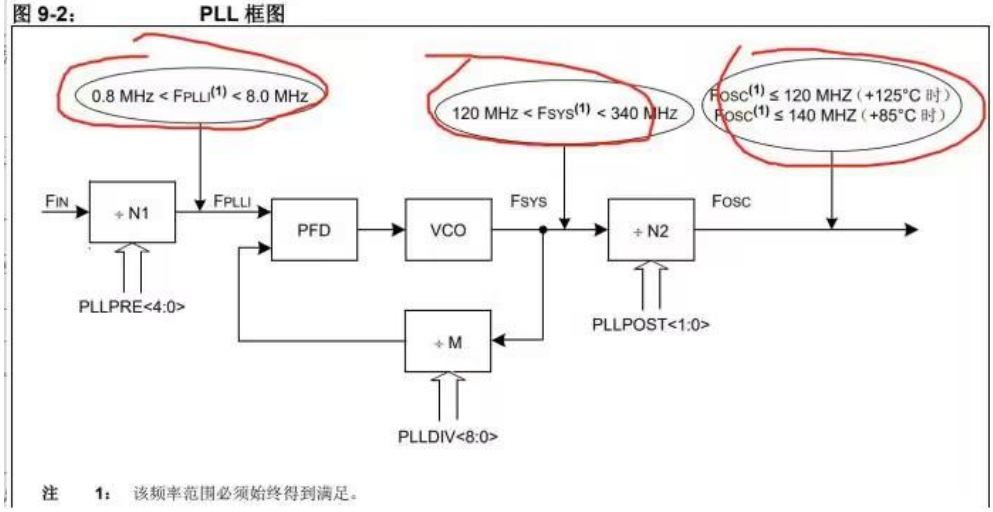

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

图 9-2：　　PLL 框图




其中，PLL 模块的框图呢，就如上图所示。我们需要注意的是，框图中有的分频及倍频部分得到的时钟信号的频率不能超出范围。换句话说，就是要合理的选择分频或者倍频的系数，来满足频率范围要求。

图 9-2：　　PLL 框图




主要的限制呢，就是三个圈内。也就是说，N1　M　N2 三个系数，需要被合理的选择。FIN 的话，我们选择的是 FRC 振荡器，如果频率不调整，则它对应的频率呢是 7.37MHz。所以，这个 N1 的设定，就参考 7.37MHz 来选择了，满足范围的系数，均可。

公式 9-2：　　FOSC 计算

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV + 2)}{(PLLPRE + 2) \times 2(PLLPOST + 1)}\right)$$

其中：

$N1 = PLLPRE + 2$

$N2 = 2 \times (PLLPOST + 1)$

$M = PLLDIV + 2$

所以，根据这个公式，我们现在没有对 M　N1　N2 进行配置，就需要知道它的默认值。这样，才能清楚 Fosc 是多少。然后，才能知道 Fcy 是多少。最终，才能确定单周期指令的时钟。这是我们的目的啊，大家不能忘记。而　N1　N2　M 系数，是靠振荡器寄存器来配置的。所以，对应的默认值呢，我们可以查看寄存器描述。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

寄存器 9-2： CLKDIV：时钟分频比寄存器

| R/W-0 | R/W-0 | R/W-1 | R/W-1 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| ROI | DOZE<2:0>(1) | | | DOZEN(2,3) | FRCDIV<2:0> | | |
| bit 15 | | | | | | | bit 8 |

| R/W-0 | R/W-1 | U-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| PLLPOST<1:0> | | — | PLLPRE<4:0> | | | | |
| bit 7 | | | | | | | bit 0 |

bit 4-0 **PLLPRE<4:0>**：PLL 相位检测器输入分频比选择位（也表示为“N1”，PLL 预分频比）
11111 = 输入被 33 分频
•
•
•
00001 = 输入被 3 分频
00000 = 输入被 2 分频（默认）

大家看，这个寄存器的低 5 位，是设置 N1 系数的。默认不设置的话，N1=2。

bit 7-6 **PLLPOST<1:0>**：PLL VCO 输出分频比选择位（也表示为“N2”，PLL 后分频比）
11 = 输出被 8 分频
10 = 保留
01 = 输出被 4 分频（默认）
00 = 输出被 2 分频
bit 5 **未实现**：读为 0

然后，bit6~bit7 是设置 N2 系数的。默认呢，N2=4。

寄存器 9-3： PLLFBD：PLL 反馈分频比寄存器

| U-0 | U-0 | U-0 | U-0 | U-0 | U-0 | U-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| — | — | — | — | — | — | — | PLLDIV<8> |
| bit 15 | | | | | | | bit 8 |

| R/W-0 | R/W-0 | R/W-1 | R/W-1 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| PLLDIV<7:0> | | | | | | | |
| bit 7 | | | | | | | bit 0 |

| 图注： | | | |
|---|---|---|---|
| R = 可读位 | W = 可写位 | U = 未实现位，读为 0 | |
| -n = POR 时的值 | 1 = 置 1 | 0 = 清零 | x = 未知 |

bit 15-9 **未实现**：读为 0
bit 8-0 **PLLDIV<8:0>**：PLL 反馈分频比位（也表示为“M”，PLL 倍频比）
111111111 = 513
•
•
•
000110000 = 50（默认）
•
•
•
000000010 = 4
000000001 = 3
000000000 = 2

然后，这个寄存器对应的是 M 系数的设置。默认呢，M=50。

公式 9-2： Fosc 计算

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2) \times 2(PLLPOST+1)}\right)$$

其中：
$N1 = PLLPRE + 2$
$N2 = 2 \times (PLLPOST + 1)$
$M = PLLDIV + 2$

所以，根据这个公式，就可以把默认的 Fosc 给计算出来了，
Fosc = 7.37MHz * 50/(2*4) = 46.0625MHz。

问： N1=PLLPRE+2?

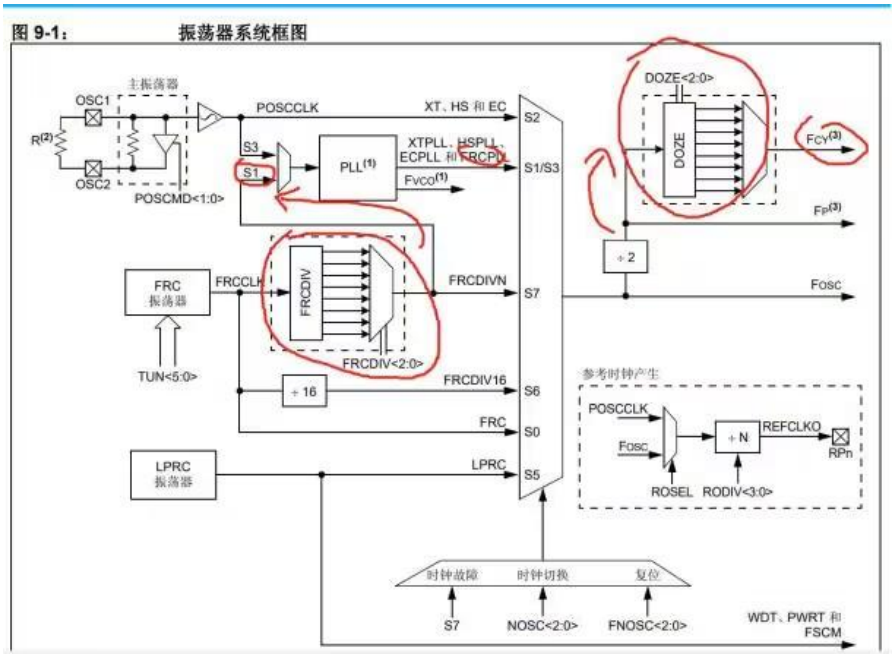

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

bit 4-0 **PLLPRE<4:0>**：PLL 相位检测器输入分频比选择位（也表示为“N1”，PLL 预分频比）
11111 = 输入被 33 分频
•
•
•
00001 = 输入被 3 分频
00000 = 输入被 2 分频（默认）

答：大家看，这个寄存器的低 5 位，是设置 N1 系数的。默认不设置的话，N1 = 2

PLLPRE=0 时， N1=0+2。

问：刚才的三个值只是寄存器默认值代表的值？
答：是的，我们现在就是算的默认值。

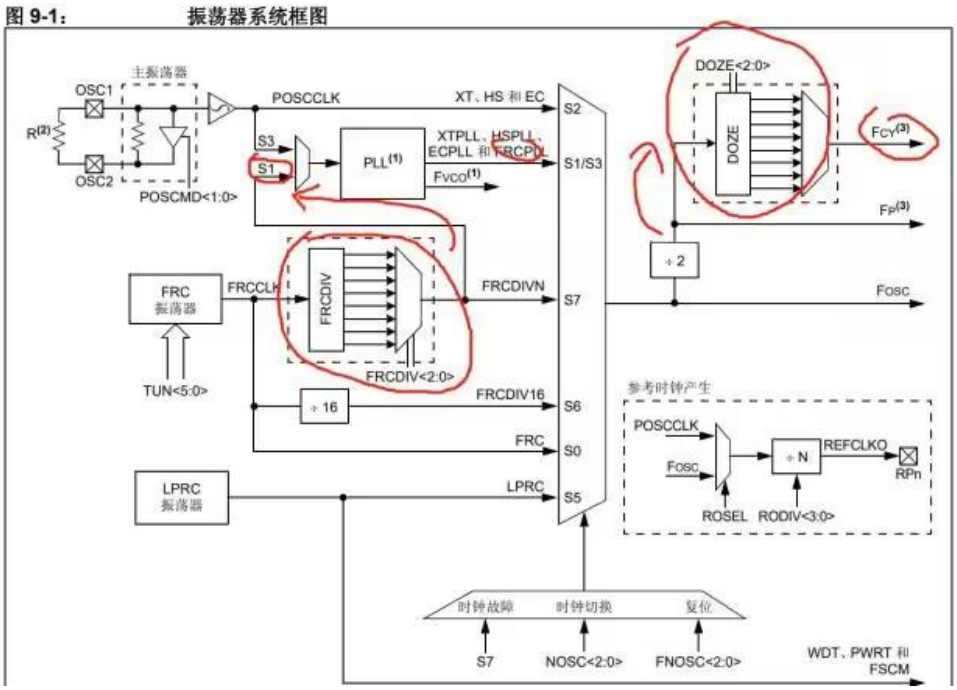



我们还要注意的是 S1 是由 FRC 来的没错。但是，中间有一个 FRCDIV 分频模块。S1 是不是 7.37MHz，需要确定 FRCDIV 分频系数。

bit 10-8 **FRCDIV<2:0>**：内部快速 RC 振荡器后分频比位
111 = FRC 被 256 分频
110 = FRC 被 64 分频
101 = FRC 被 32 分频
100 = FRC 被 16 分频
011 = FRC 被 8 分频
010 = FRC 被 4 分频
001 = FRC 被 2 分频
000 = FRC 被 1 分频（默认）
bit 7-6 PLLPOST<1:0>：PLL VCO 输出分频比选择位（也表示为“N

而根据寄存器的描述，FRCDIV 就是确定的 1 分频。所以，S1 确定的就是 7.37MHz 了。

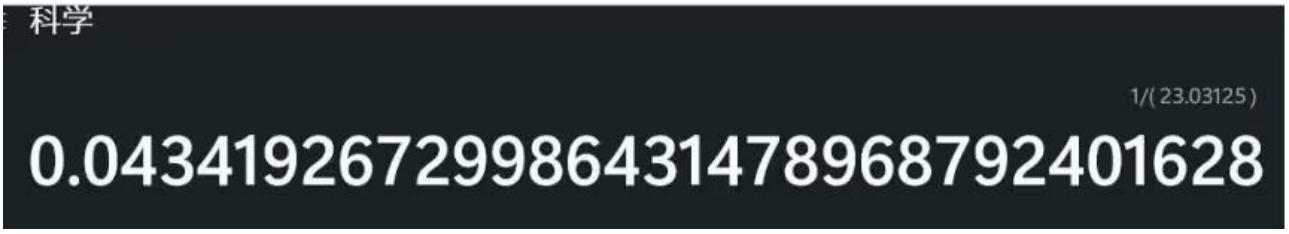

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。




第二个，Fosc 确定了的话，Fcy 又是多少？ Fcy 的来源呢，是 Fosc/2. 但是中间又经过了一个 DOZE 模块。也相当于支持分频。所以，我们还得确定一下 DOZE 的默认值。



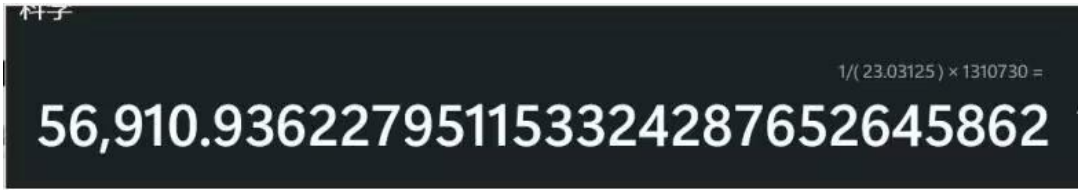

虽然呢，DOZE 是默认 8 分频。但是呢，这个功能默认是没有使能的，强制被设置为了 1 分频。所以，Fcy = Fosc/2. 最终，Fcy = 46.0625MHz/2 = 23.03125MHz。这下，我们就可以把单周期指令的执行时间给计算出来了。




这个数值的单位是 us。这个值，乘一下之前的指令周期数，就可以得到延时代码对应的延时时间。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

1+4+65536*4+（65535*4+1）*4+4+（3*4+1）=5+262144+（262140+1）*4+4+13=1310730（指令周期）。这是我们之前计算出来的指令周期。




计算出来，延时的时间是　56.910ms。

```
mov     #4,W1   ;1个指令周期
clr     W0      ;1个指令周期 * 4
dec     W0,W0   ;1个指令周期 * 65536 * 4
bra     NZ,$-2  ;未跳转是1个指令周期，跳转是4个指令周期 (65535 * 4 + 1) * 4
dec     W1,W1   ;1个指令周期 * 4
bra     NZ,$-8  ;未跳转是1个指令周期，跳转是4个指令周期  3*4 + 1    1310730 *   1/23.03125MHz = 56.91093622795ms
```

所以，到目前为止，这段延时代码对应的延时时间，也就清楚了。本身这段代码没有太多的技术含量，搞清背后的原理才是重要的。

我们就继续启动文件了

```
mov     #4,W1   ;1个指令周期
clr     W0      ;1个指令周期 * 4
dec     W0,W0   ;1个指令周期 * 65536 * 4
bra     NZ,$-2  ;未跳转是1个指令周期，跳转是4个指令周期 (65535 * 4 + 1) * 4
dec     W1,W1   ;1个指令周期 * 4
bra     NZ,$-8  ;未跳转是1个指令周期，跳转是4个指令周期  3*4 + 1    1310730 *   1/23.03125MHz = 56.91093622795ms

mov     #__SP_init,W15          ;SP stack begin
mov     #__SPLIM_init,W0
mov     W0,SPLIM                ;SP stack end
```

启动文件中，另一个功能是设置堆栈的大小。堆栈的话，是用于保存现场数据的。例如，在发生子程序调用，中断的时候，会先把之前的数据存到堆栈中。等子程序调用或者中断结束，再恢复之前的数据，以便于程序继续执行。所以，既然要保存数据的话，就需要设置存储空间大小。

```
mov     #__SP_init,W15          ;SP stack begin
mov     #__SPLIM_init,W0
mov     W0,SPLIM                ;SP stack end
```

第一条指令呢，是把堆栈的起始地址赋值给 W15 寄存器。W15 寄存器比较特殊，它就是专用作为堆栈指针的寄存器。

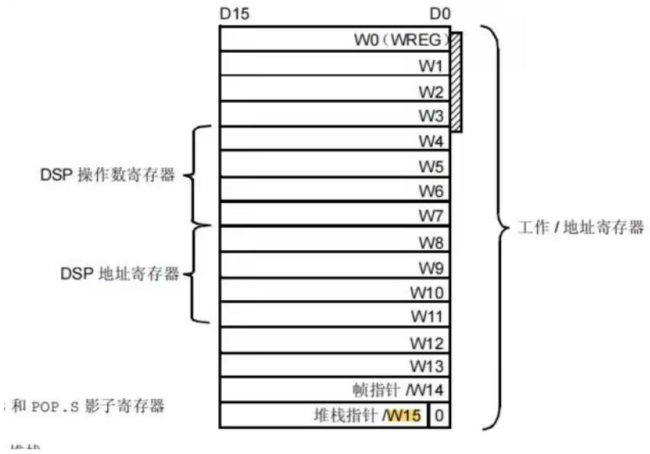

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

### 3.1 寄存器

dsPIC33EPXXXGP50X、dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC20X 器件在编程模型中有 16 个 16 位工作寄存器。每个工作寄存器都可以充当数据、地址或地址偏移量寄存器。第 16 个工作寄存器（**W15**）作为软件堆栈指针工作，用于中断和调用。

**XGP50X、 dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC**

**编程模型**

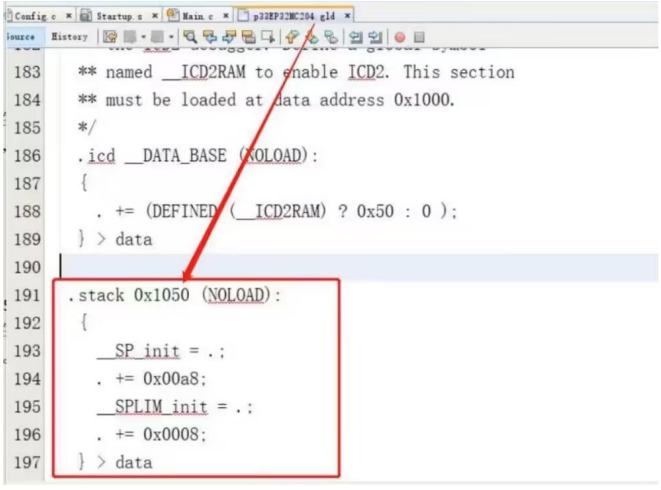



### 4.4.4 软件堆栈

**W15** 寄存器作为专用的软件堆栈指针（Stack Pointer，SP），并被异常处理、子程序调用和返回自动修改；但是，**W15** 可以被任何指令以与所有其他 W 寄存器相同的方式引用。这样就简化了对堆栈指针的读、写和操作（例如，创建堆栈帧）。

也就是说，这个 W15 寄存器中，存储的就是当前的堆栈地址。那么，这个堆栈的起始地址是多少呢？这里的话，就来自于链接描述文件中的堆栈空间的设置了。

大家看下，这段 gld 中的代码，是我们自己设置的。它的作用呢，就是设置堆栈空间，以及溢出缓冲空间。堆栈空间的起始地址呢，就是 RAM 中的 0x1050 地址。





本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
.stack 0x1050 (NOLOAD):
 {
   __SP_init = .;
   . += 0x00a8;
   __SPLIM_init = .;
   . += 0x0008;
 } > data
```

这句话，把"."赋值给__SP_init，"."就表示 0x1050 这个地址。然后，堆栈的空间是多大呢？

```
.stack 0x1050 (NOLOAD):
 {
   __SP_init = .;
   . += 0x00a8;
   __SPLIM_init = .;
   . += 0x0008;
 } > data
```

就是在 0x1050 的基础上，加上 0x00a8 大小的地址空间。也就是 0x1050 ~0x10F8 这段地址空间，就是堆栈空间。

问：芯片的堆栈是上压的吗？
答：这个芯片是向上的。

大小就是 0xa8 = 168 字节，也就是说，我们设置的堆栈空间，可以最多保存 168 个字节的数据。实际上，已经比较大了。那么，为了防止堆栈溢出，出现堆栈溢出错误，其实，我们还设置了一段溢出缓冲空间。

```
.stack 0x1050 (NOLOAD):
 {
   __SP_init = .;
   . += 0x00a8;
   __SPLIM_init = .;
   . += 0x0008;
 } > data
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

溢出空间就是 8 个字节。也就是在堆栈的结束地址的基础上，再增加 8 个字节的空间，作为溢出限制。这样的话，堆栈空间就设置好了。

```
mov     #__SP_init,W15      ;SP stack begin
mov     #__SPLIM_init,W0
mov     W0,SPLIM            ;SP stack end
```

那么，我们启动文件中呢，就是对设置的堆栈起始地址跟堆栈溢出空间的限制地址进行设置。

| SPLIM | 0 |
|---|---|

堆栈指针限制

写到对应的寄存器中。这样，堆栈的设置就好了。

问：栈顶是 1050，堆栈首地址是 10f8？

4.4.4 软件堆栈

W15 寄存器作为专用的软件堆栈指针（Stack Pointer，SP），并被异常处理、子程序调用和返回自动修改；但是，W15 可以被任何指令以与所有其他 W 寄存器相同的方式引用。这样就简化了对堆栈指针的读、写和操作（例如，创建堆栈帧）。

注： 为了防止不对齐的堆栈访问，W15<0> 被硬件固定设置为 0。

所有复位均将 W15 初始化为 0x1000。该地址确保在所有 dsPIC33EPXXXGP50X、dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC20X 器件中，SP 指向有效的 RAM 并允许不可屏蔽陷阱异常使用堆栈。这些可能在 SP 被用户软件初始化之前发生。在初始化期间，可以将 SP 再编程以指向数据空间内的任何存储单元。

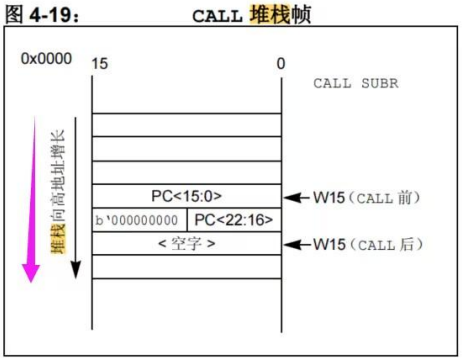
堆栈指针总是指向第一个可用的空字并从低地址到高地址填充软件堆栈。图 4-19 显示了它如何在弹出堆栈（读）时预递减，以及在压入堆栈（写）时后递增。

当 PC 压入堆栈时，PC<15:0> 被压入第一个可用的堆栈字，然后 PC<22:16> 被压入第二个可用的堆栈单元。对于任何 CALL 指令执行期间的 PC 压栈，压栈前 PC 的 MSB 被零扩展，如图 4-19 所示。异常处理期间，PC 的 MSB 与 CPU 状态寄存器 SR 的低 8 位组合。这样就能在中断处理期间自动保存 SRL 的内容。

注 1：为了维持系统堆栈指针（W15）一致性，W15 永远不会受（EDS）分页影响，因此被限制在地址范围 0x0000 至 0xFFFF。当 W14 用作堆栈帧指针（SFA = 1）时，这同样适用于 W14。

2：由于堆栈可置于 X 和 Y 空间中并能访问这两个空间，因此在使用堆栈、特别是涉及到 C 语言开发环境中的局部自动变量时必须小心。

图 4-19： CALL 堆栈帧




答：
堆栈地址是从低到高生长的。

问：这个和 STM32 在 Ram 顶部设定为堆栈起始地址不一样，这样的好处是什么呢？
答：单片机堆栈生长方向不一样，这是单片机设计者决定的。

问：. +=0x00a8 呢?
答：表示当前地址 + 0x00a8 的字节长度。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

### 3.6.4 特殊点符号

特殊符号 “.” 指代正在汇编的当前地址。因此，表达式：

```
melvin:  .word .      ; in a data section
```

会将 melvin 定义为包含它自己的数据地址。给 . 赋值的效果和 .org 伪指令相同。因此，表达式：

```
. = .+2
```

等同于：

```
.org .+2
```

接受符号 “$” 作为 “.” 的同义符号。

当在可执行代码段中使用时，“.” 指代 PC 地址。在 16 位器件上，每个指令字会使 PC 递增 2。不允许奇数值。

问：NOLOAD 呢？

#### 9.7.5.10 输出段类型

每个输出段都有一个类型。类型为在圆括号中的关键字。定义了以下类型：

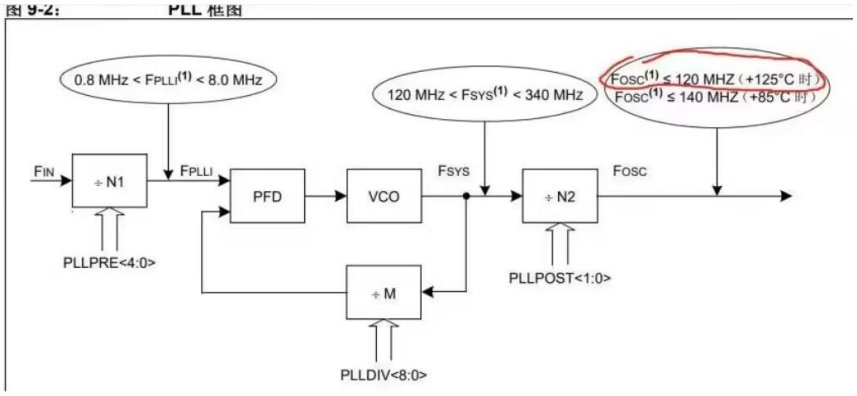
NOLOAD

答：该段应该被标记为不可装载，因此当运行程序时，它将不被装载到存储器中。

```
mov     #__SP_init,W15          ;SP stack begin
mov     #__SPLIM_init,W0
mov     W0,SPLIM                ;SP stack end   堆栈的设置

mov     #0x0000,W0
mov     W0,CLKDIV
mov     #0x0024,W0
mov     W0,OSCTUN
mov     #0x0040,W0
mov     W0,PLLFBD
```

堆栈设置好之后，下面的一个功能就是设置时钟。因为默认的 Fosc=46.0625MHz，而我们单片机呢，最高支持 120MHz 的 Fosc。所以，我们需要对 N1 N2 M 进行配置，使得 Fosc = 120MHz。也就是说，这段代码就是对 N1 N2 M 进行配置的。





本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看，因为 120MHz 适用的温度范围宽。所以，就设置 Fosc 最高为 120MHz 了。当然，如果温度要求不高，那么也可以设置到 140MHz。咱们这里呢，就以 120MHz 为例来讲解啊。

实际上，N1　N2　M 的配置，使得 Fosc 为 120MHz 的方式可能有很多种啊。大家呢，也可以自己尝试按照你自己的参数配置一下。我这里呢，就以我之前用到的配置为例，来给大家讲解一下哈。

```
35    mov     #__SP_init,W15          ;SP stack begin
36    mov     #__SPLIM_init,W0
37    mov     W0,SPLIM                ;SP stack end   堆栈的设置
38
39    mov     #0x0000,W0
40    mov     W0,CLKDIV
41    mov     #0x0024,W0
42    mov     W0,OSCTUN
43    mov     #0x0040,W0
44    mov     W0,PLLFBD
45
```

先看前两句。

```
mov     #0x0000,W0
mov     W0,CLKDIV
mov     #0x0024,W0
mov     W0,OSCTUN
mov     #0x0040,W0
mov     W0,PLLFBD
```

红框的这两条语句，目的就是实现 CLKDIV=0x0000。

**寄存器 9-2：　CLKDIV：时钟分频比寄存器**

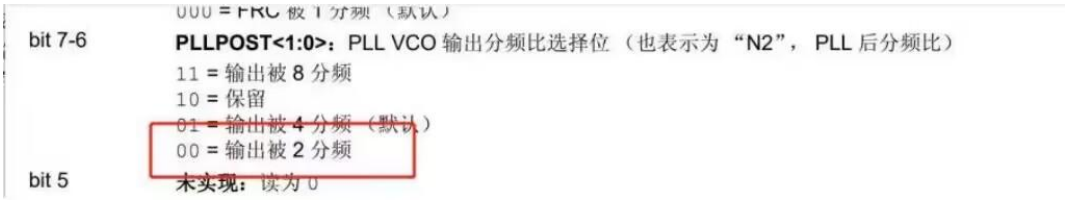
| R/W-0 | R/W-0 | R/W-1 | R/W-1 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| ROI | DOZE<2:0>(1) | | | DOZEN(2,3) | FRCDIV<2:0> | | |
| bit 15 | | | | | | | bit 8 |

| R/W-0 | R/W-1 | U-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| PLLPOST<1:0> | | — | PLLPRE<4:0> | | | | |
| bit 7 | | | | | | | bit 0 |

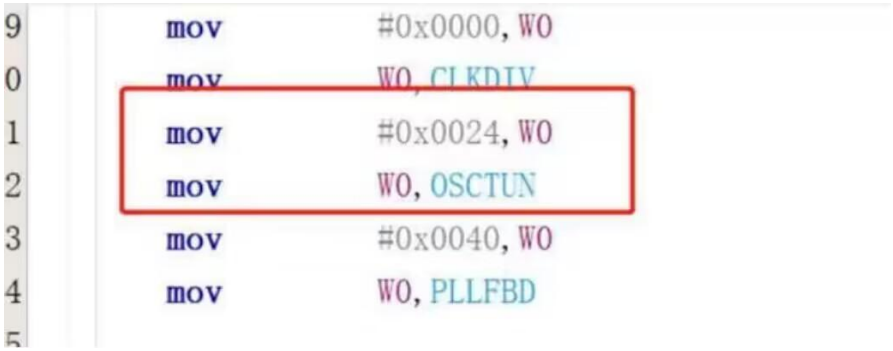
bit 4-0　**PLLPRE<4:0>：**PLL 相位检测器输入分频比选择位（也表示为“N1”，PLL 预分频比）
11111 = 输入被 33 分频
•
•
•
00001 = 输入被 3 分频
00000 = 输入被 2 分频（默认）


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

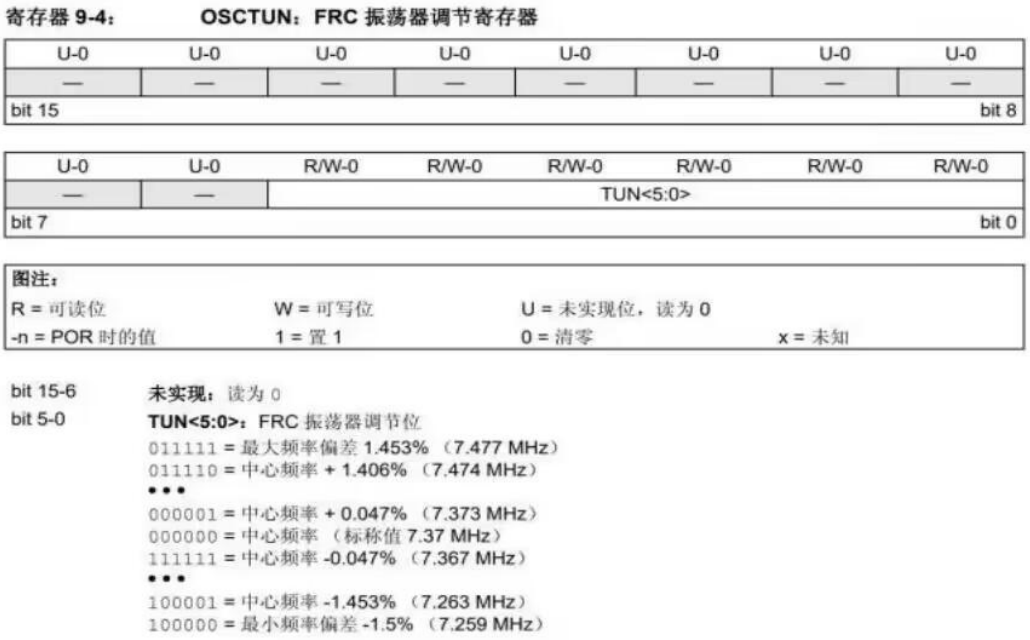
寄存器的低 5 位是设置 N1 系数的，设置为 0 的话，就是 N1=2。




然后呢，bit7~bit6 是 N2 系数的。设置为 0 的话，就是 N2=2。这个 CLKDIV 寄存器。就是侧重于设置 N1N2 两个系数。




接着呢，这两条语句是实现设置 OSCTUN=0x0024。我们可以来了解一下 OSCTUN 寄存器的作用。




大家看下，寄存器的名称是 FRC 振荡器调节寄存器。也就是说，这个寄存器的作用是用来调节 FRC 振荡器的频率的。标准是 7.37MHz，但是可以用这个寄存器进行微调。大家看，很显然这 6 个位都是 0 的时候，对应的频率是 7.37MHz。然后，往上或者往下，是对应这个


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

频率加或者减。大家看下，最高位是 0 或者 1，来区分振荡器频率是加或者减。最高位是 1 的话，就是中心频率往下减。最高位是 0 的话，就是中心频率往上增。

```
mov     #0x0000, W0
mov     W0, CLKDIV
mov     #0x0024, W0
mov     W0, OSCTUN
mov     #0x0040, W0
mov     W0, PLLFBD
```

然后，我们设置的是 十六进制的 0x24，转换为二进制就是 0b 0010 0100，很显然，最高位是 1。所以呢，我们的调整是中心频率往下减，也就是把频率往小调整。

| | |
|---|---|
| bit 15-6 | **未实现：**读为 0 |
| bit 5-0 | **TUN<5:0>：**FRC 振荡器调节位 |
| | 011111 = 最大频率偏差 1.453% （7.477 MHz） |
| | 011110 = 中心频率 + 1.406% （7.474 MHz） |
| | • • • |
| | 000001 = 中心频率 + 0.047% （7.373 MHz） |
| | 000000 = 中心频率 （标称值 7.37 MHz） |
| | 111111 = 中心频率 -0.047% （7.367 MHz） |
| | • • • |
| | 100001 = 中心频率 -1.453% （7.263 MHz） |
| | 100000 = 最小频率偏差 -1.5% （7.259 MHz） |

那么，我们忽略最高位的话，总共可以调节的挡位就是 32 档。那么，我们配置的低 5 位是 4，也就是总共下降的挡位就是 32-4=28。然后，每下降一个挡位，中心频率就减小 0.047%。所以，减小 28 个挡位的话，中心频率就是减小 28*0.047%。所以，减小的频率百分比就是 28* 0.047/100 = 0.01316。然后用 0.01316 * 7.37MHz，就是减小的频率。也就是调整后的频率就是 7.37MHz - 0.01316 * 7.37MHz = 7.2730108MHz。

```
mov     #0x0000, W0
mov     W0, CLKDIV
mov     #0x0024, W0
mov     W0, OSCTUN          ;频率调整为7.2730108MHz
mov     #0x0040, W0
mov     W0, PLLFBD
```

所以，这两句话的作用，就是调整 FRC 的频率为 7.2730108MHz。

```
90  mov     #0x0000, W0
91  mov     W0, CLKDIV
92  mov     #0x0024, W0
93  mov     W0, OSCTUN          ;频率调整为7.2730108MHz
94  mov     #0x0040, W0
95  mov     W0, PLLFBD
```

最后一个是设置 M 系数的寄存器。


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

寄存器 9-3：　PLLFBD：PLL 反馈分频比寄存器

| U-0 | U-0 | U-0 | U-0 | U-0 | U-0 | U-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| — | — | — | — | — | — | — | PLLDIV<8> |
| bit 15 | | | | | | | bit 8 |

| R/W-0 | R/W-0 | R/W-1 | R/W-1 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
|---|---|---|---|---|---|---|---|
| PLLDIV<7:0> | | | | | | | |
| bit 7 | | | | | | | bit 0 |

| 图注： | | | |
|---|---|---|---|
| R = 可读位 | W = 可写位 | U = 未实现位，读为 0 | |
| -n = POR 时的值 | 1 = 置 1 | 0 = 清零 | x = 未知 |

bit 15-9　**未实现：**读为 0

bit 8-0　**PLLDIV<8:0>：**PLL 反馈分频比位（也表示为 “M”，PLL 倍频比）

111111111 = 513
•
•
•
000110000 = 50（默认）
•
•
•
000000010 = 4
000000001 = 3
000000000 = 2

那么，我们这里设置为 0x40 也就是十进制的 64，则对应的 M=64+2=66。所以，N1　N2　M　以及调整后的目标频率都清楚。自然，最终倍频之后的　Fosc 也就可以计算了。

公式 9-2：　Fosc 计算

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2)\times 2(PLLPOST+1)}\right)$$

其中：

$N1 = PLLPRE + 2$

$N2 = 2 \times (PLLPOST + 1)$

$M = PLLDIV + 2$

这是公式，Fosc　=　7.2730108　*　（66/2*2）　=　120.0046782MHz。基本上就是 120MHz 了。

```
mov     #0x0000,W0
mov     W0,CLKDIV           ;N1 = 2  N2 = 2
mov     #0x0024,W0
mov     W0,OSCTUN           ;频率调整为7.273010SMHz
mov     #0x0040,W0
mov     W0,PLLFBD           ;M = 66   Fosc = 7.273010SMHz * (66/2*2) = 120.0046782MHz
```

这个呢，就是这段代码的作用啊。

我问大家，为什么要配 120MHz，不这样配行不行，你们会这样配置吗？

大家课后想一下，我们今天内容就到这里。大家晚安，早点休息！


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。